Proposed presentation structure

1. The team: Introduction (20 seconds)

Briefly introduce the four members.

---

2. Assignment 1 — 50 seconds

Description: A dispatch console that helps operators perform calculations and operational checks using information entered directly into the program.

Running 8:

2.1 Introduction: The menu works smoothly but we want to make more emphasis in service 8 since it gives us a broader vision about hte complexity of the dispatcher.

2.2 Description: Service 8 compares delivery prices for three service levels. We enter a distance and weight, and the program calculates each price and identifies the cheapest and most expensive options.

2.3 Relevancy: If we look at the code, we reuse the same `calculate_quote()` function three times, changing only the service code. This avoids repeating the calculation and keeps the logic consistent. That's the main design principle behind this application: reusable functions that work together through a single console.

---

3. Assignment 2: 1 minute 10 seconds

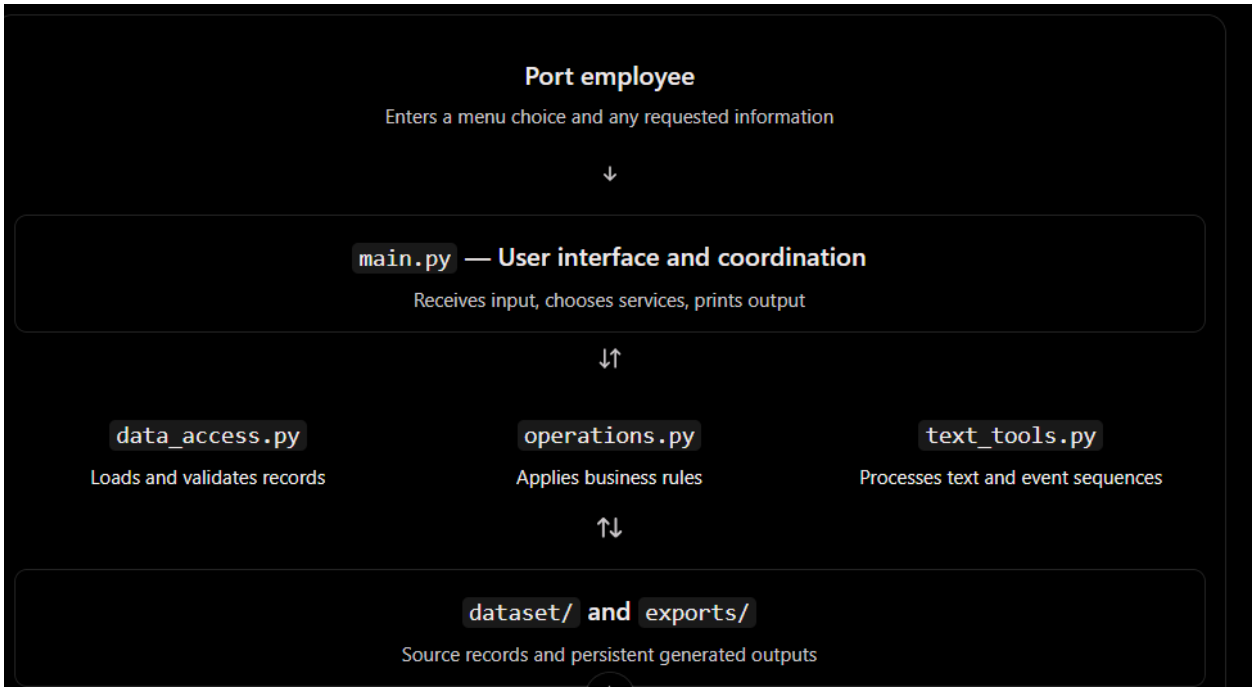
3.1. Description

A port intelligence console that helps port operators retrieve, organize and analyze operational information about vessels, shipments, port calls, incidents and weather conditions using JSON and CSV datasets.

3.2. Introduction and architecture

Unlike Assignment 1, Assignment 2 requires us to process external operational data from JSON and CSV files. We organized the application into four modules, separating the user interface, data access, business logic and text processing.

This allows different services to reuse shared functions while keeping their responsibilities separate.



3.3 Demonstration — Service 5: Find port calls by month

- Select service 5 and enter a month.
- Show the resulting port-call report.
- Explain how the application loads vessel and port-call records, connects them using vessel IDs, validates arrival dates, filters by month and sorts the results.

3.4 Show the relevant code

- `main.py` — `run_port_calls_by_month()`
- `data_access.py` — `load_port_calls()` and `load_vessels()`
- `operations.py` — `collect_month_calls()`

Main point: The service demonstrates how independently organized modules collaborate to produce a single operational report.

4. Service 5: Boundary / Error demonstration — 40 seconds

Scenario: Enter an invalid month, such as `13`, followed by a valid month, such as `10`.

Expected behavior: The console rejects the invalid month, displays `Error - Enter a month from 1 to 12.`, and repeats only the month prompt without restarting the service.

Code to show: `main.py` → `run_port_calls_by_month()` — the `while`, `try/except` and month-range validation.

Test evidence: Open `test-ledger.csv` and locate the Task 5 invalid-month and recovery tests.

Main point: The application validates user input and allows the operator to recover without interrupting the program.

5. Examiner questions — 60 seconds

Reserve this minute for questions. Keep the source files open and be prepared to explain:

- Functions, parameters, arguments and return values.
- Input validation, conditions, loops and error handling.
- JSON/CSV loading, relationships between records and sorting.
- Regex, sliding-window algorithm, cached search index and persistent outputs.

All four members should be prepared to explain code beyond their individual contributions.